# The Story of the Paper

Slicing Quantum Programs for Comprehension: What a Quantum Slice Should Look Like

Big-picture document for the rewrite, 28 October 2026

## What the paper is about

A developer who sees a qubit end in the wrong state asks which lines of the program determine it. For classical programs, program slicing answers that question. This paper asks what a quantum slice must keep to answer it correctly, which dependence rule produces such a slice, and how the slice should be shown on the developer's own source code.

## The problem, and why it matters for comprehension

Quantum programs are hard to comprehend. Their intermediate states cannot be inspected without disturbing them, and a gate on one qubit can change another, as in phase kickback, where a gate changes the qubit that controls it. Bug detectors such as LintQ and QChecker find known patterns, but many faults are wrong outputs that no pattern describes, and a slice that tells the developer where to look is the natural aid. Slicing has been proposed for quantum programs, but its dependence rules have never been checked against what a quantum program computes. That matters for comprehension more than for any other use, because a slice that hides code able to change the qubit misleads the reader, who will not look where the fault is. And such slices look right. Reading a CNOT the classical way, with its control unchanged, gives wrong slices for 26.0% of 784 circuit–qubit pairs, and 43% of those wrong slices reproduce the program's outcome histogram, so a developer who checks a slice by its outcomes would accept them.

**Thesis.** A slice supports comprehension only if it never hides code that can change the qubits the developer asks about. Classical rules violate this, often invisibly, and entanglement-based rules add code that cannot matter. One rule satisfies it, the *write-all rule*, under which a gate writes every qubit it touches. It is proved sound and dependence-complete and costs milliseconds.

## Contributions

1. **Semantics.** A definition of what a slice of a set of qubits must preserve, for every input, and of when an operation is a dependence of those qubits.
2. **One rule suffices.** Under the write-all rule the quantum dependence graph (QDG) gives slices that are sound, with the induction mechanized in Isabelle/HOL, dependence-complete and distributive over criteria, in at most 7 ms per criterion. Three kinds of dependence are all a slice needs, classical data, classical control and quantum data dependence. Entanglement is neither necessary nor sufficient for dependence, and what looks like quantum control dependence is quantum data dependence. For each of the 21 standard two-qubit gates, dropping either qubit from the write set misses a dependence.
3. **What the alternatives cost.** A pre-registered study on 784 pairs and 5,688 seeded-fault units measures every alternative against the rule. All but 10 of the 3,174 dependences the classical PDG misses come from its single choice of write set. The rules nest, the dependence set is itself a sound slice at the study's input, and the QDG adds only 4.8 points to it, equalling it on 559 pairs, while the QPDG's entanglement edges add 2.7 times as much code that cannot matter. Pruning by simulating one input makes slices 23.2% smaller but correct for that input only.
4. **Delivery for developers.** srcSlice-Q is a static slicer, which analyses the source of a program without running it. It shows a slice as source lines, loop iterations, the classical lines that compute them and a profile of each qubit,

for Qiskit programs and OpenQASM 2 circuits. It traces 4,182 of LintQ's 7,568 GitHub programs and 95% of the expert-written Qiskit HumanEval programs that build a circuit, in 131 ms per program in the median, and a qubit's slice holds 24% of a program's code lines.

5. **An honest limit.** Slices narrow where a developer looks but do not improve mutation-based fault ranking, which motivates a study with developers.

## What is new, and what we build on

**New.** The semantic justification of a rule as a slicing rule, with its proofs; the measurement of what every alternative costs against it; the finding that three kinds of dependence suffice and the proposed quantum kinds add none; the classical PDG's failure and why it is invisible to an outcome check; and the quantum layer of srcSlice-Q, which gives srcSlice's slice profiles quantum roles, a static trace of qubits and loop iterations, the QDG and qubit profiles.

**Built on.** The relation itself is not new. It is the backward light cone that compilers and Qiskit's `LightCone` pass compute, and LintQ orders operations by it to find bug patterns; we establish it as the slicing rule rather than invent it. Locality of quantum operations (no-signaling) carries the soundness proof. Classical slicing and the program dependence graph give the vocabulary. The QPDG of Yu and Zhao is reimplemented as a documented baseline. srcML and srcSlice give srcSlice-Q its source representation and its classical lines, and Behler et al.'s work on srcML for quantum languages names the qubit slicer as future work. Bugs4Q, MQT Bench, QASMBench, LintQ's corpus and warnings, Qiskit HumanEval and published mutation operators supply the programs and faults.

## How the sections fit together

The introduction states the question and the thesis. A motivating example shows a classical slice that reproduces the outcomes and hides the fault. The core section defines what a slice must keep, states the write-all rule with its guarantees, and places every other rule relative to it. srcSlice-Q is described next, briefly, as the way the rule reaches a developer; its full design is the subject of a separate tool paper. The pre-registered study design and its results follow, organized by four research questions, which rules are sound and complete (RQ1), how much smaller a slice can safely be (RQ2), what each rule does to fault localization (RQ3), and what each costs (RQ4). The discussion turns the results into implications for comprehension tools. Related work, threats to validity and the conclusion close the paper.

**Decisions to check.** (i) Section order, with related work near the end, as above, or second. (ii) How much of srcSlice-Q stays in this paper, given the separate tool paper; above it is one contribution and one short section. (iii) The name of the rule, "write-all rule".